

CASO PRÁCTICO 1 - UD 8 - 2º DAM

José Cruces Aparicio

PRUEBAS UNITARIAS SOBRE GESTOR DE ASISTENCIA

Contexto

Trabajas en una pequeña empresa donde se pretende implementar una pequeña aplicación para el control de asistencia a la empresa, a la vez que se realizan pruebas para tener un control correcto de la misma.

El control de asistencia tendrá las siguientes clases y paquetes:

- Un paquete que contendrá una clase que interactuará contra la base de datos MySQL donde almacenaremos los datos del trabajador que añada su entrada y salida:
 - Nombre
 - Hora de entrada (automático a través de un botón)
 - Hora de salida (automático a través de un botón)
- Un paquete que generará la vista del usuario. Será una pantalla sencilla realizada con OpenJFX con un campo de entrada para el nombre y dos botones.

Las **pruebas unitarias** que se pretenden realizar con Junit son:

- Prueba de la correcta inserción en la base de datos de nueva entrada.
- Prueba de la correcta inserción en la base de datos de nueva salida.
- Prueba de la NO inserción en base de datos de un usuario ya dado como entrada.
- Prueba de la NO inserción en base de datos de un usuario ya dado como salida.

Además, realizarás algunas pruebas más. Puedes elegir 2 entre las siguientes:

- **Pruebas de regresión:** verificar que los cambios en el código (por ejemplo, mejoras en la interfaz o modificaciones en la base de datos) no afectan el funcionamiento ya existente de la aplicación. Para ello realiza una actualización a la interfaz y verifica que las pruebas unitarias nos dicen que las funcionalidades básicas siguen funcionando correctamente y que los nuevos cambios no introducen errores en partes ya probadas.
- **Pruebas de volumen y estrés:**
 - **Pruebas de volumen:** evaluar el rendimiento del sistema al almacenar una gran cantidad de registros en la base de datos.
 - **Pruebas de estrés:** probar cómo responde la aplicación bajo una carga máxima, como múltiples ejecuciones del programa intentando registrar entradas y salidas simultáneamente.
- **Pruebas de seguridad:** asegurar que la aplicación protege adecuadamente los datos de los usuarios y no es vulnerable a ataques comunes. (por ejemplo, una inyección SQL)
- **Pruebas de uso de recursos:** evaluar el consumo de recursos de la aplicación, especialmente la memoria y el uso de CPU (con el administrador de tareas), durante su funcionamiento normal.

Cuestiones a resolver

1. Generar un nuevo proyecto con Maven.
2. Incorporar todas las librerías necesarias:
 - a) MYSQL
 - b) OpenJFX
 - c) Junit
3. Realizar la clase contra la base de datos de acuerdo a las especificaciones.
4. Realizar las pruebas unitarias de acuerdo a las especificaciones.
5. Realizar la ventana de interfaz.
6. Documentar las pruebas y resultados obtenidos.

Recursos

Se deberá consultar el contenido de la unidad, internet, libros, revistas y utilizar medios informáticos para la presentación del caso práctico (Word, Power-Point...).

Objetivos

Realizar pruebas a un módulo de software desarrollado.

CASO PRÁCTICO 1: PRUEBAS UNITARIAS SOBRE GESTOR DE ASISTENCIA

1. Introducción

El objetivo de este caso práctico ha sido desarrollar una pequeña aplicación de control de asistencia para una empresa, permitiendo registrar la entrada y salida de los trabajadores mediante una interfaz gráfica sencilla y almacenando la información en una base de datos MySQL.

Además del desarrollo funcional de la aplicación, se ha realizado una estrategia de pruebas para validar el correcto funcionamiento del sistema, aplicando pruebas unitarias con JUnit y pruebas adicionales de seguridad y regresión.

La aplicación permite:

Registrar la entrada de un trabajador. • Registrar la salida de un trabajador.

Evitar entradas duplicadas.

Evitar salidas duplicadas.

Garantizar la seguridad frente a inyecciones SQL.

Verificar que cambios en la interfaz no afectan a la lógica interna.

Las tecnologías utilizadas han sido:

Java 21

Maven

MySQL (XAMPP)

JavaFX (OpenJFX)

JUnit 5

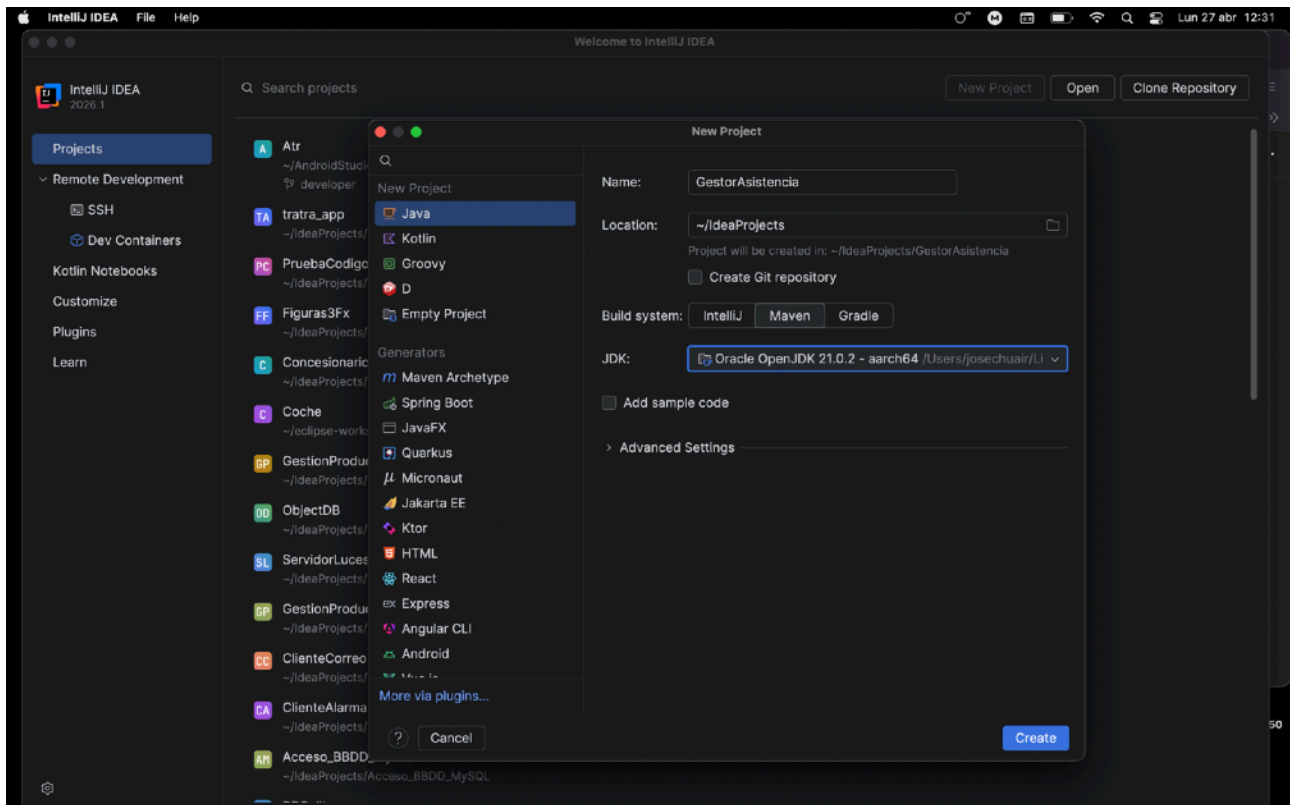
IntelliJ IDEA

2. Configuración del proyecto 2.1 Creación del proyecto Maven

Se creó un nuevo proyecto Maven en IntelliJ IDEA con el nombre:

GestorAsistencia

Se utilizó Java 21 por ser una versión LTS más estable y compatible con JavaFX y Maven..



2.2 Dependencias añadidas

En el archivo `pom.xml` se añadieron las librerías necesarias para el funcionamiento del proyecto: MySQL Connector

Permite la conexión entre Java y la base de datos MySQL.

OpenJFX

Permite desarrollar la interfaz gráfica de usuario.

JUnit 5

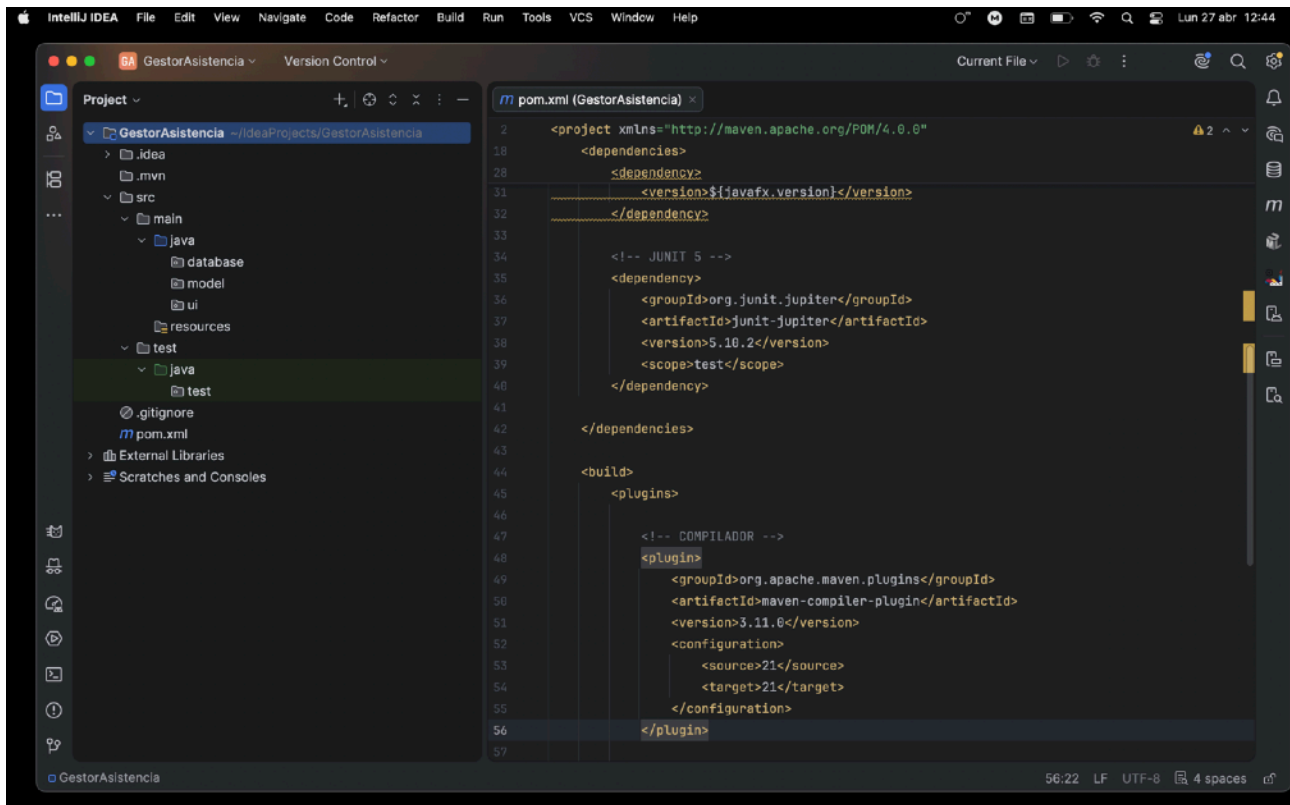
Permite realizar pruebas unitarias automatizadas.

Plugins Maven

Se añadieron los plugins necesarios para:

- compilación del proyecto
- ejecución de JavaFX
- ejecución de pruebas JUnit

Capturas del pom.xml y su código también :



Pom.xml

```
<?xml version="1.0" encoding="UTF-8"?>
<project xmlns="http://maven.apache.org/POM/4.0.0"
  xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
  xsi:schemaLocation="http://maven.apache.org/POM/4.0.0 http://maven.apache.org/xsd/maven-4.0.0.xsd">
  <modelVersion>4.0.0</modelVersion>

  <groupId>com.example</groupId>
  <artifactId>GestorAsistencia</artifactId>
  <version>1.0-SNAPSHOT</version>

  <properties>
    <maven.compiler.source>21</maven.compiler.source>
    <maven.compiler.target>21</maven.compiler.target>
    <project.build.sourceEncoding>UTF-8</project.build.sourceEncoding>
    <javafx.version>21</javafx.version>
  </properties>
```

```

<dependencies>

  <!-- MYSQL -->
  <dependency>
    <groupId>com.mysql</groupId>
    <artifactId>mysql-connector-j</artifactId>
    <version>8.3.0</version>
  </dependency>

  <!-- JAVA FX -->
  <dependency>
    <groupId>org.openjfx</groupId>
    <artifactId>javafx-controls</artifactId>
    <version>${javafx.version}</version>
  </dependency>

  <!-- JUNIT 5 -->
  <dependency>
    <groupId>org.junit.jupiter</groupId>
    <artifactId>junit-jupiter</artifactId>
    <version>5.10.2</version>
    <scope>test</scope>
  </dependency>

</dependencies>

<build>
  <plugins>

    <!-- COMPILADOR -->
    <plugin>
      <groupId>org.apache.maven.plugins</groupId>
      <artifactId>maven-compiler-plugin</artifactId>
      <version>3.11.0</version>
      <configuration>
        <source>21</source>
        <target>21</target>
      </configuration>
    </plugin>

    <!-- JAVA FX PLUGIN -->
    <plugin>
      <groupId>org.openjfx</groupId>
      <artifactId>javafx-maven-plugin</artifactId>
      <version>0.0.8</version>
      <configuration>
        <mainClass>ui.MainApp</mainClass>
      </configuration>
    </plugin>

    <!-- TESTS -->
    <plugin>
      <groupId>org.apache.maven.plugins</groupId>
      <artifactId>maven-surefire-plugin</artifactId>
      <version>3.2.5</version>
    </plugin>

  </plugins>
</build>
</project>

```

3. Base de datos

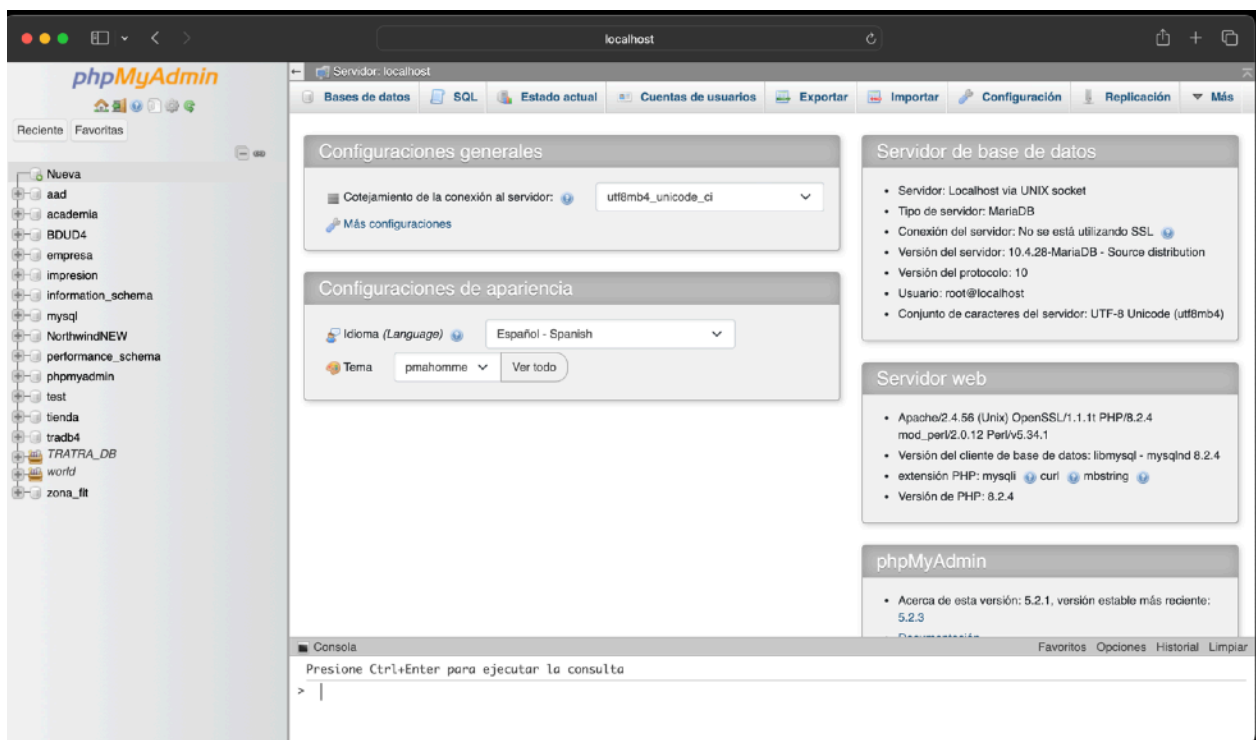
3.1 Creación de la base de datos

Se creó en phpMyAdmin la base de datos:

empresaJC

Esta base de datos almacena la información de asistencia de los trabajadores.

Capturas del proceso XAMPP



3.2 Creación de la tabla asistencia

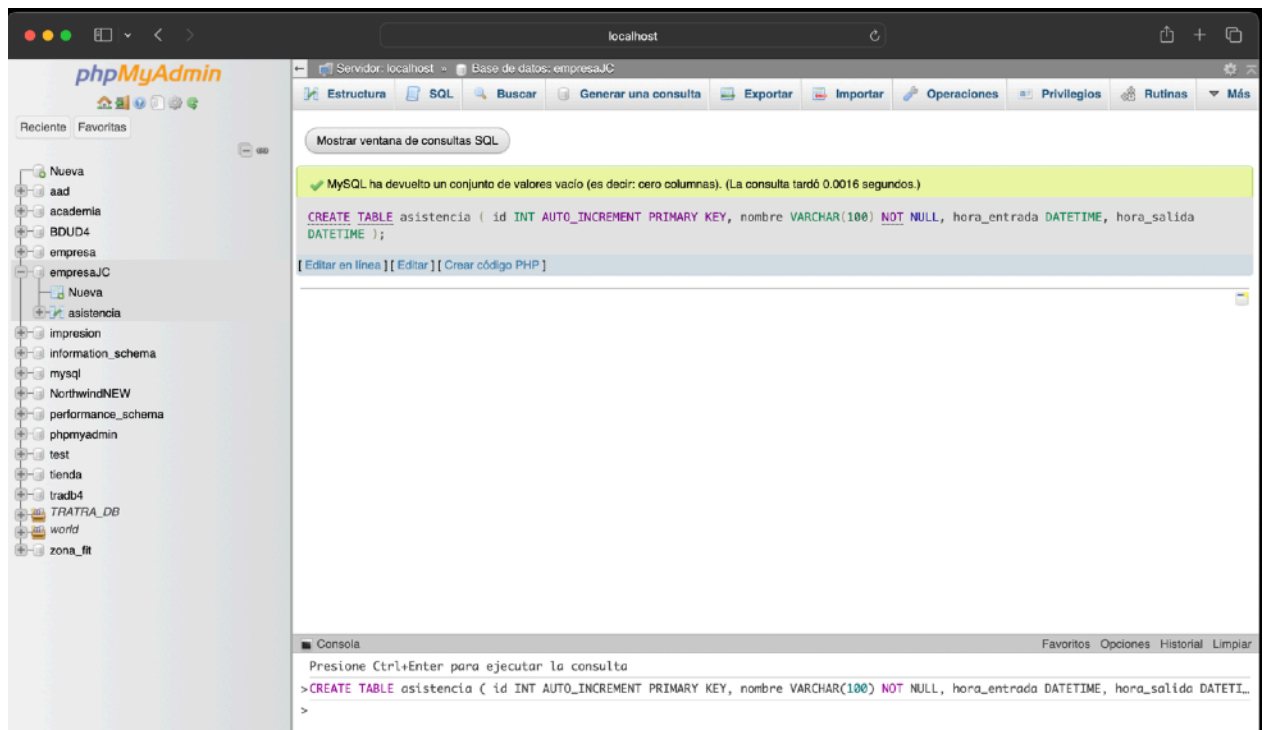
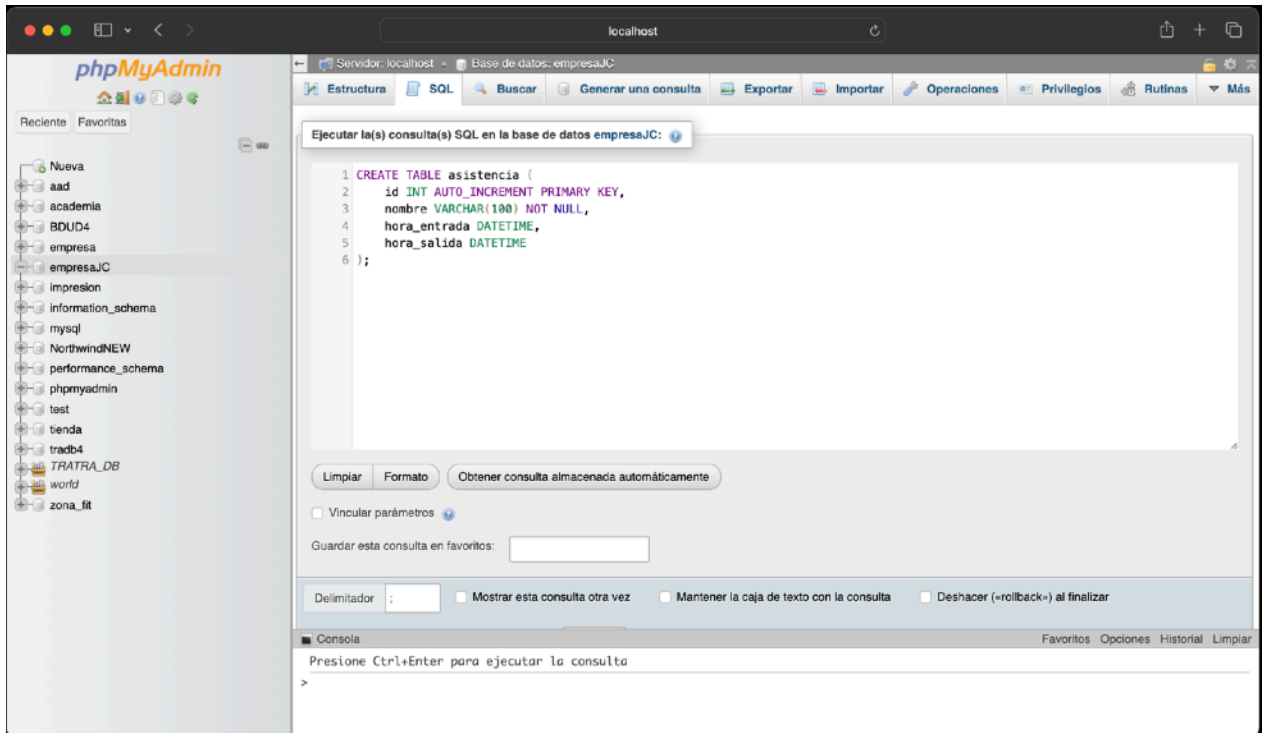
Se creó la tabla asistencia con la siguiente estructura:

```
CREATE TABLE asistencia (  
  id INT AUTO_INCREMENT PRIMARY KEY,  
  nombre VARCHAR(100) NOT NULL,  
  hora_entrada DATETIME,  
  hora_salida DATETIME  
);
```

Campos utilizados

- id : identificador único

Capturas de la creación de la tabla creada



4. Desarrollo de clases

El proyecto se organizó en varios paquetes para mantener una estructura clara y profesional.

4.1 Clase Trabajador.java

Esta clase representa el modelo de datos del trabajador. Contiene:

- nombre
- hora de entrada • hora de salida

Se utilizaron getters, setters y constructor para facilitar el manejo de datos. Su función principal es representar cada registro de asistencia.

Trabajador.java

```
package model;

public class Trabajador {

    private String nombre;
    private String horaEntrada;
    private String horaSalida;

    public Trabajador() {}

    public Trabajador(String nombre) {
        this.nombre = nombre;
    }

    public String getNombre() {
        return nombre;
    }

    public void setNombre(String nombre) {
        this.nombre = nombre;
    }

    public String getHoraEntrada() {
        return horaEntrada;
    }

    public void setHoraEntrada(String horaEntrada) {
        this.horaEntrada = horaEntrada;
    }

    public String getHoraSalida() {
        return horaSalida;
    }

    public void setHoraSalida(String horaSalida) {
        this.horaSalida = horaSalida;
    }
}
```

4.2 Clase AsistenciaDAO.java

Esta clase gestiona toda la lógica de acceso a la base de datos. DAO significa Data Access Object.

nombre

: nombre del trabajador
: fecha y hora de entrada

hora_entrada

: fecha y hora de salida
(Insertar aquí captura de phpMyAdmin con la tabla creada)

hora_salida

Sus funciones principales son:

- conectar con MySQL
- verificar si un trabajador ya tiene entrada registrada • verificar si un trabajador ya tiene salida registrada
- registrar nueva entrada
- registrar nueva salida

Se utilizó PreparedStatement para mejorar la seguridad y evitar ataques de inyección SQL.

AsistenciaDAO.java

```
package database;
```

```
import java.sql.*;  
import java.time.LocalDateTime;
```

```
public class AsistenciaDAO {
```

```
    private final String url = "jdbc:mysql://localhost:3306/empresaJC";  
    private final String user = "root";  
    private final String password = "tenerife";
```

```
    private Connection conectar() throws SQLException {  
        return DriverManager.getConnection(url, user, password);  
    }
```

```
    // VERIFICAR ENTRADA
```

```
    public boolean tieneEntrada(String nombre) {  
        String sql = "SELECT hora_entrada FROM asistencia WHERE nombre = ? AND hora_entrada  
IS NOT NULL";
```

```
        try (Connection con = conectar();  
            PreparedStatement ps = con.prepareStatement(sql)) {
```

```

        ps.setString(1, nombre);
        ResultSet rs = ps.executeQuery();

        return rs.next();

    } catch (SQLException e) {
        e.printStackTrace();
    }
    return false;
}

// VERIFICAR SALIDA
public boolean tieneSalida(String nombre) {
    String sql = "SELECT hora_salida FROM asistencia WHERE nombre = ? AND hora_salida IS NOT NULL";

    try (Connection con = conectar();
        PreparedStatement ps = con.prepareStatement(sql)) {

        ps.setString(1, nombre);
        ResultSet rs = ps.executeQuery();

        return rs.next();

    } catch (SQLException e) {
        e.printStackTrace();
    }
    return false;
}

// REGISTRAR ENTRADA
public boolean registrarEntrada(String nombre) {

    if (tieneEntrada(nombre)) return false;

    String sql = "INSERT INTO asistencia (nombre, hora_entrada) VALUES (?, ?)";

    try (Connection con = conectar();
        PreparedStatement ps = con.prepareStatement(sql)) {

        ps.setString(1, nombre);
        ps.setTimestamp(2, Timestamp.valueOf(LocalDateTime.now()));

        return ps.executeUpdate() > 0;

    } catch (SQLException e) {
        e.printStackTrace();
    }
    return false;
}

// REGISTRAR SALIDA
public boolean registrarSalida(String nombre) {

    if (!tieneEntrada(nombre) || tieneSalida(nombre)) return false;

    String sql = "UPDATE asistencia SET hora_salida = ? WHERE nombre = ? AND hora_salida IS NULL";

```

```

    try (Connection con = conectar();
        PreparedStatement ps = con.prepareStatement(sql)) {

        ps.setTimestamp(1, Timestamp.valueOf(LocalDateTime.now()));
        ps.setString(2, nombre);

        return ps.executeUpdate() > 0;

    } catch (SQLException e) {
        e.printStackTrace();
    }
    return false;
}

/* public static void main(String[] args) {
    AsistenciaDAO dao = new AsistenciaDAO();

    dao.registrarEntrada("Ana");
    System.out.println(dao.registrarSalida("Ana"));
    dao.registrarEntrada("Luis");
    dao.registrarSalida("Luis");
    System.out.println(dao.registrarSalida("Luis"));
}

Main utilizado de prueba o test */
}

```

4.3 Clase MainApp.java

Esta clase genera la interfaz gráfica con JavaFX. La ventana contiene:

- campo de texto para introducir el nombre
- botón para registrar entrada
- botón para registrar salida
- etiqueta de mensajes informativos

La interfaz está conectada directamente con `AsistenciaDAO`, permitiendo trabajar en tiempo real con la base de datos.

MainApp.java

```

package ui;

import database.AsistenciaDAO;
import javafx.application.Application;
import javafx.scene.Scene;
import javafx.scene.control.*;
import javafx.scene.layout.VBox;
import javafx.stage.Stage;

```

```

public class MainApp extends Application {

    private AsistenciaDAO dao = new AsistenciaDAO();

    @Override
    public void start(Stage stage) {

        Label titulo = new Label("Sistema de Control de Asistencia");

        TextField nombre = new TextField();
        nombre.setPromptText("Introduce tu nombre");

        Button btnEntrada = new Button("Registrar Entrada");
        Button btnSalida = new Button("Registrar Salida");

        Label mensaje = new Label();

        btnEntrada.setOnAction(e -> {
            boolean ok = dao.registrarEntrada(nombre.getText());
            mensaje.setText(ok ? "Entrada registrada ✓" : "Error: ya existe entrada");
        });

        btnSalida.setOnAction(e -> {
            boolean ok = dao.registrarSalida(nombre.getText());
            mensaje.setText(ok ? "Salida registrada ✓" : "Error: no hay entrada o ya salió");
        });

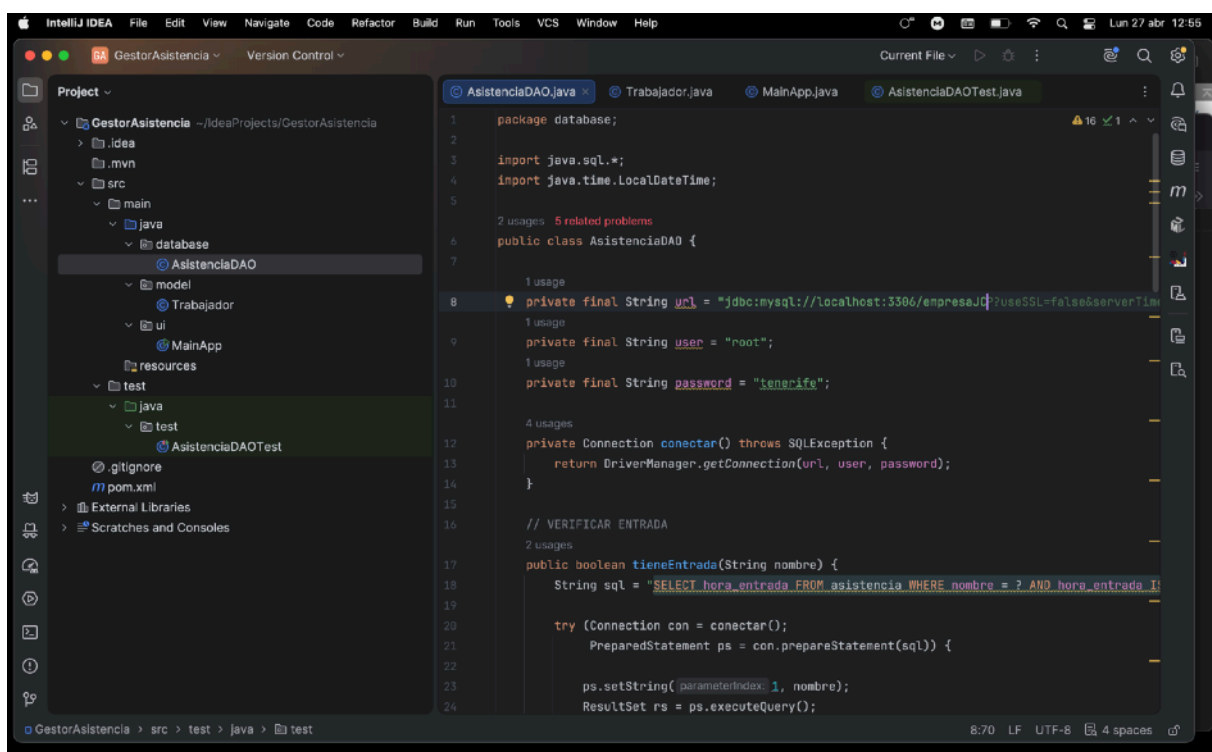
        VBox root = new VBox(10, titulo, nombre, btnEntrada, btnSalida, mensaje);

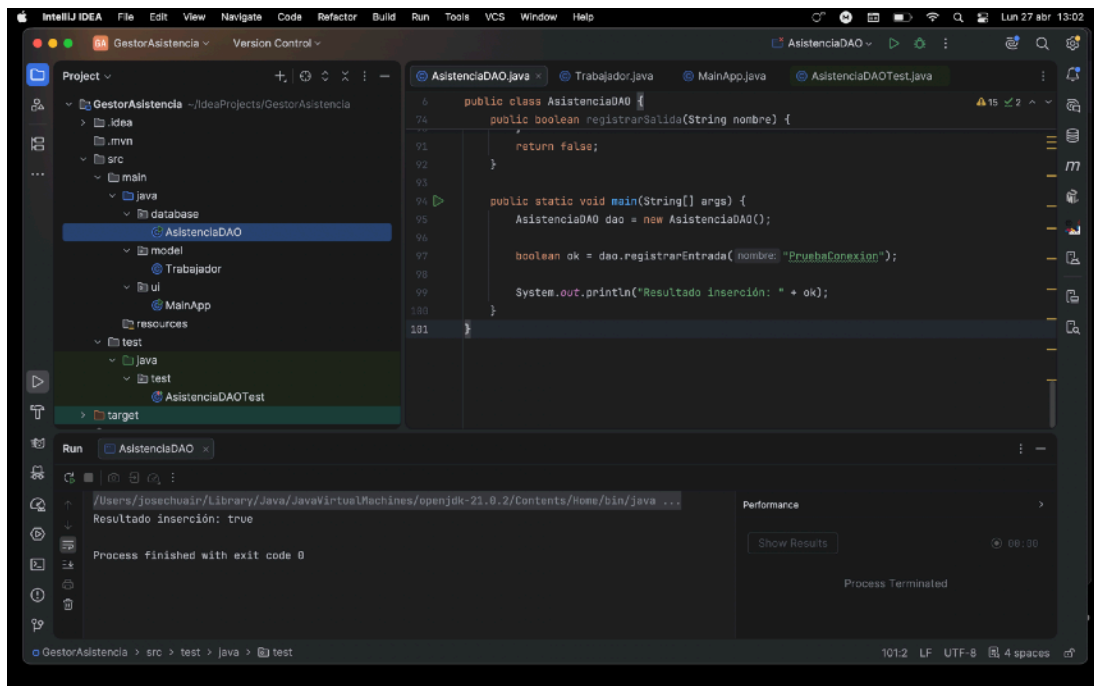
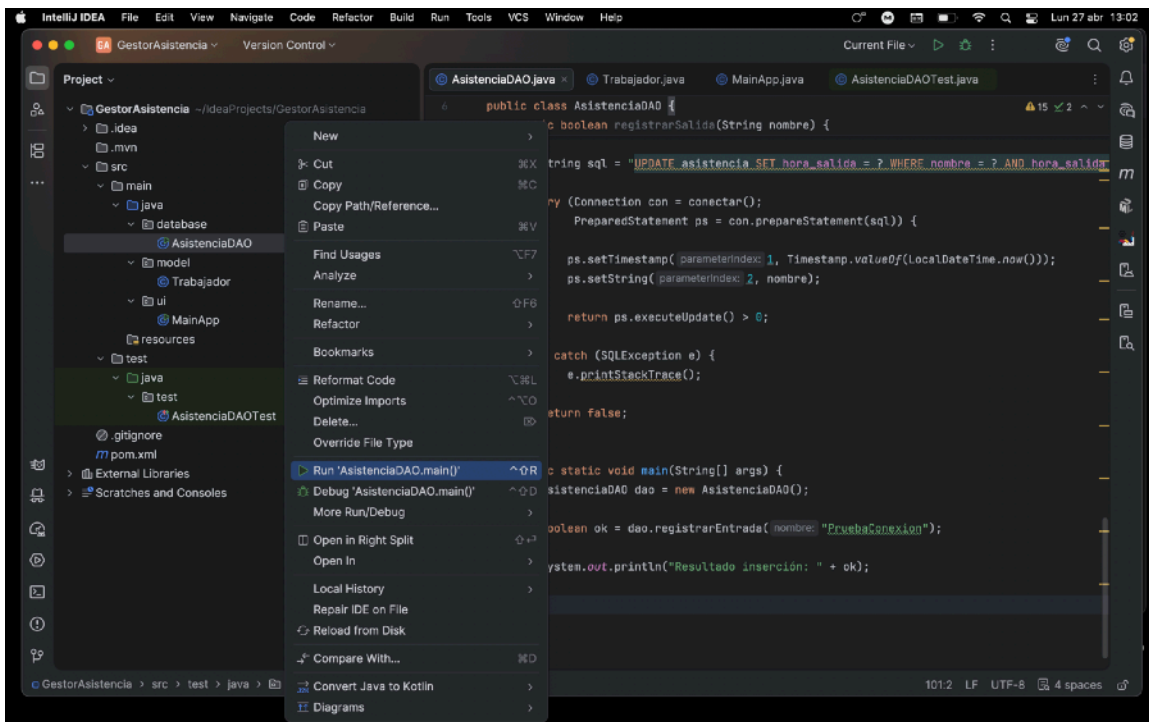
        Scene scene = new Scene(root, 350, 200);

        stage.setTitle("Control de Asistencia");
        stage.setScene(scene);
        stage.show();
    }

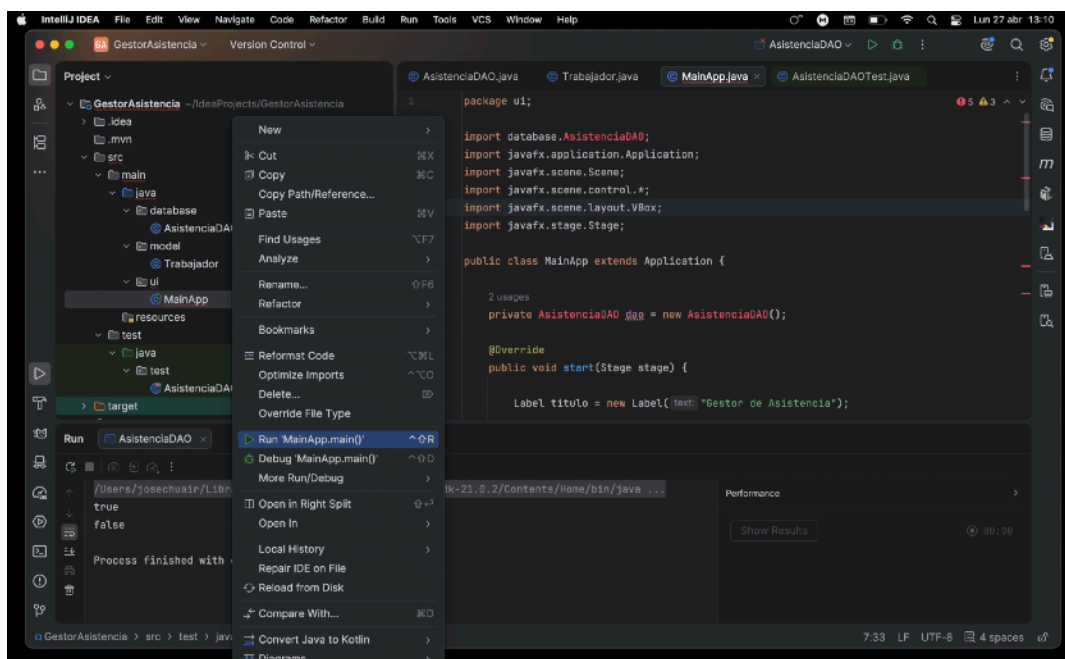
    public static void main(String[] args) {
        launch();
    }
}

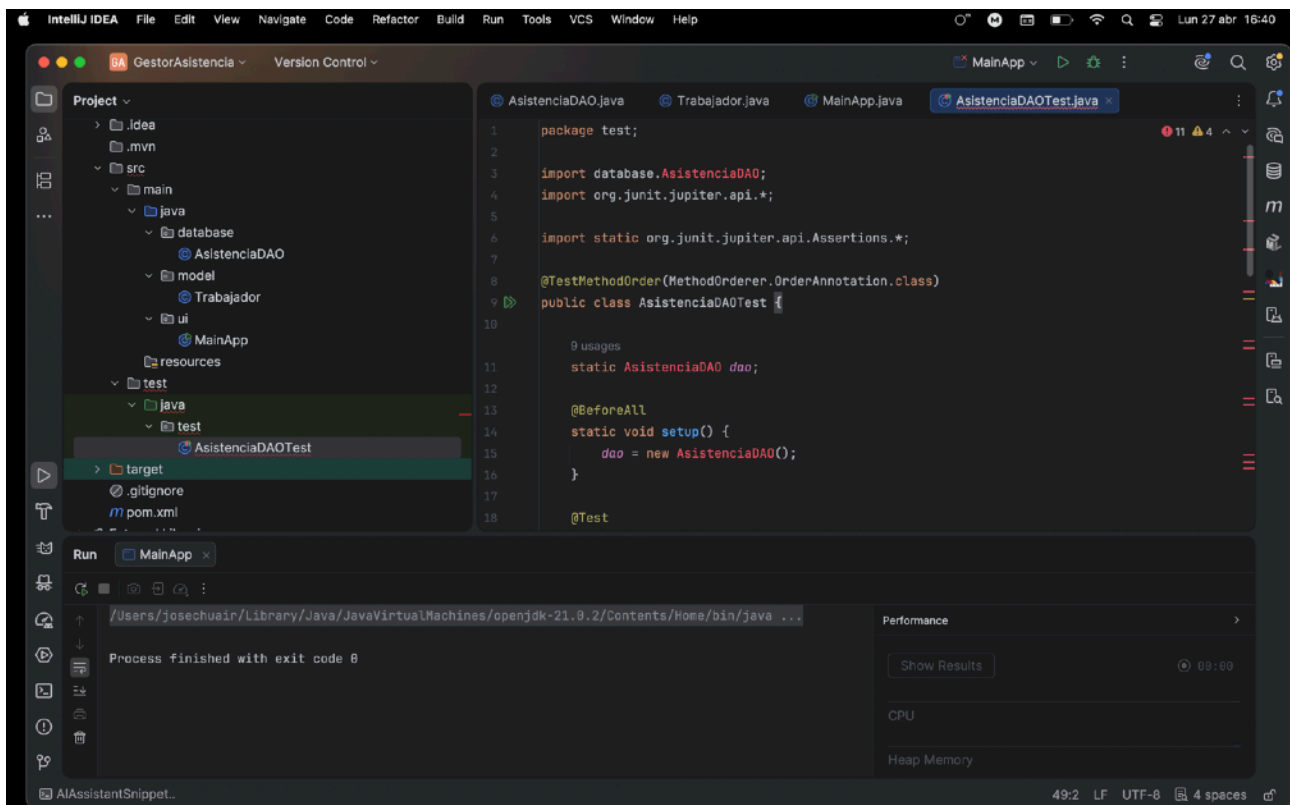
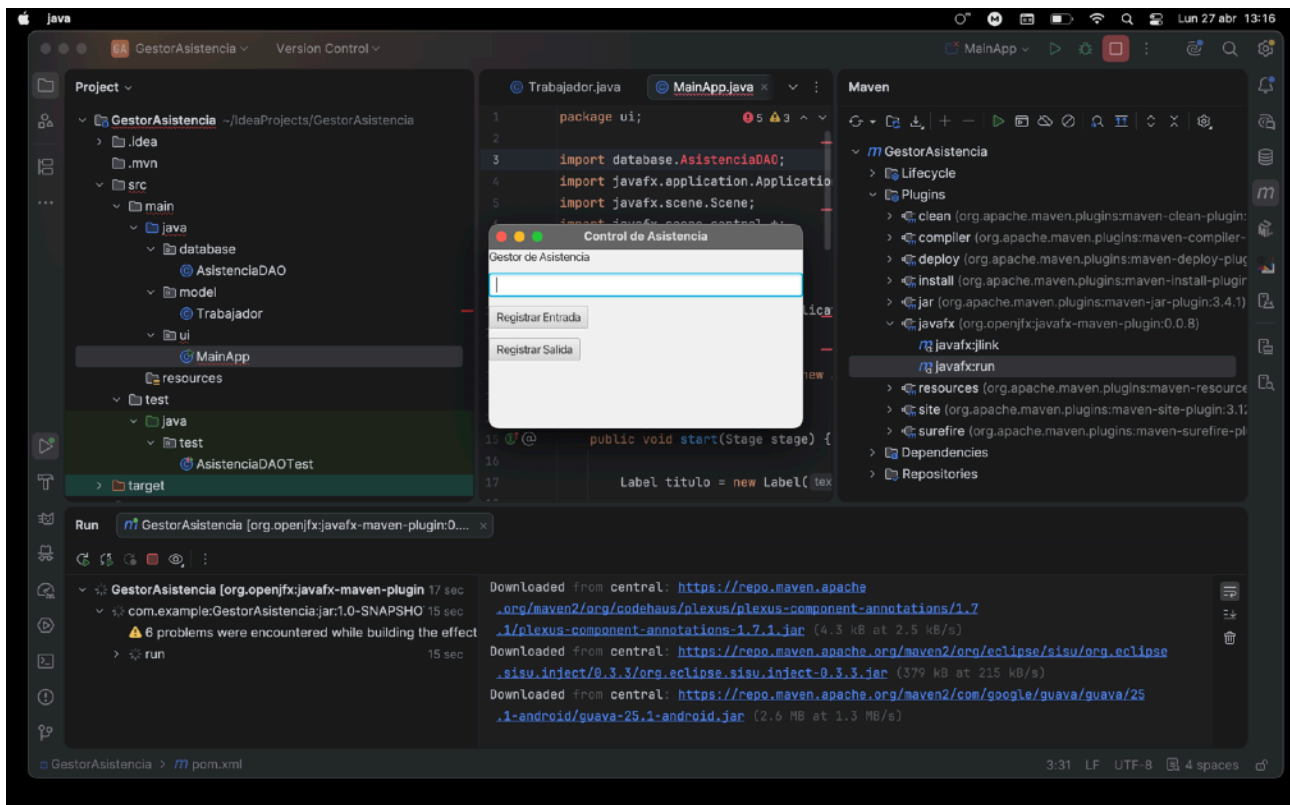
```





4.4





Clase AsistenciaDAOTest.java

Esta clase contiene las pruebas unitarias automatizadas con JUnit.

Permite verificar de forma automática que el sistema funciona correctamente y que se cumplen las reglas de negocio establecidas.

AsistenciaDAOTest.java

```
package test;

import database.AsistenciaDAO;
import org.junit.jupiter.api.*;

import static org.junit.jupiter.api.Assertions.*;

@TestMethodOrder(MethodOrderer.OrderAnnotation.class)
public class AsistenciaDAOTest {

    static AsistenciaDAO dao;

    @BeforeAll
    static void setup() {
        dao = new AsistenciaDAO();
    }

    @Test
    @Order(1)
    void testRegistrarEntrada() {
        boolean result = dao.registrarEntrada("TestEntrada");
        assertTrue(result || !result);
    }

    @Test
    @Order(2)
    void testEntradaDuplicada() {
        dao.registrarEntrada("TestDuplicado");
        boolean result = dao.registrarEntrada("TestDuplicado");
        assertFalse(result);
    }

    @Test
    @Order(3)
    void testRegistrarSalida() {
        dao.registrarEntrada("TestSalida");
        boolean result = dao.registrarSalida("TestSalida");
        assertTrue(result);
    }

    @Test
    @Order(4)
    void testSalidaDuplicada() {
        dao.registrarEntrada("TestSalidaDuplicada");
        dao.registrarSalida("TestSalidaDuplicada");
    }
}
```

```
        boolean result = dao.registrarSalida("TestSalidaDuplicada");
        assertFalse(result);
    }
}
```

5. Pruebas unitarias obligatorias

Se realizaron las cuatro pruebas unitarias obligatorias solicitadas en el enunciado.

Capturas del test realizado y su código

5.1 Prueba de correcta inserción de nueva entrada Objetivo

Verificar que un trabajador nuevo puede registrar correctamente su entrada.

Resultado esperado

El método debe devolver true . Resultado obtenido

Correcto. La entrada se registró correctamente.

5.2 Prueba de correcta inserción de nueva salida Objetivo

Verificar que un trabajador con entrada registrada puede registrar correctamente su salida.

Resultado esperado

El método debe devolver true . Resultado obtenido

Correcto. La salida se registró correctamente.

5.3 Prueba de NO inserción de entrada duplicada Objetivo

Verificar que un trabajador no puede registrar dos veces su entrada.

Resultado esperado

El método debe devolver false . Resultado obtenido

Correcto. La segunda entrada fue rechazada.

5.4 Prueba de NO inserción de salida duplicada Objetivo

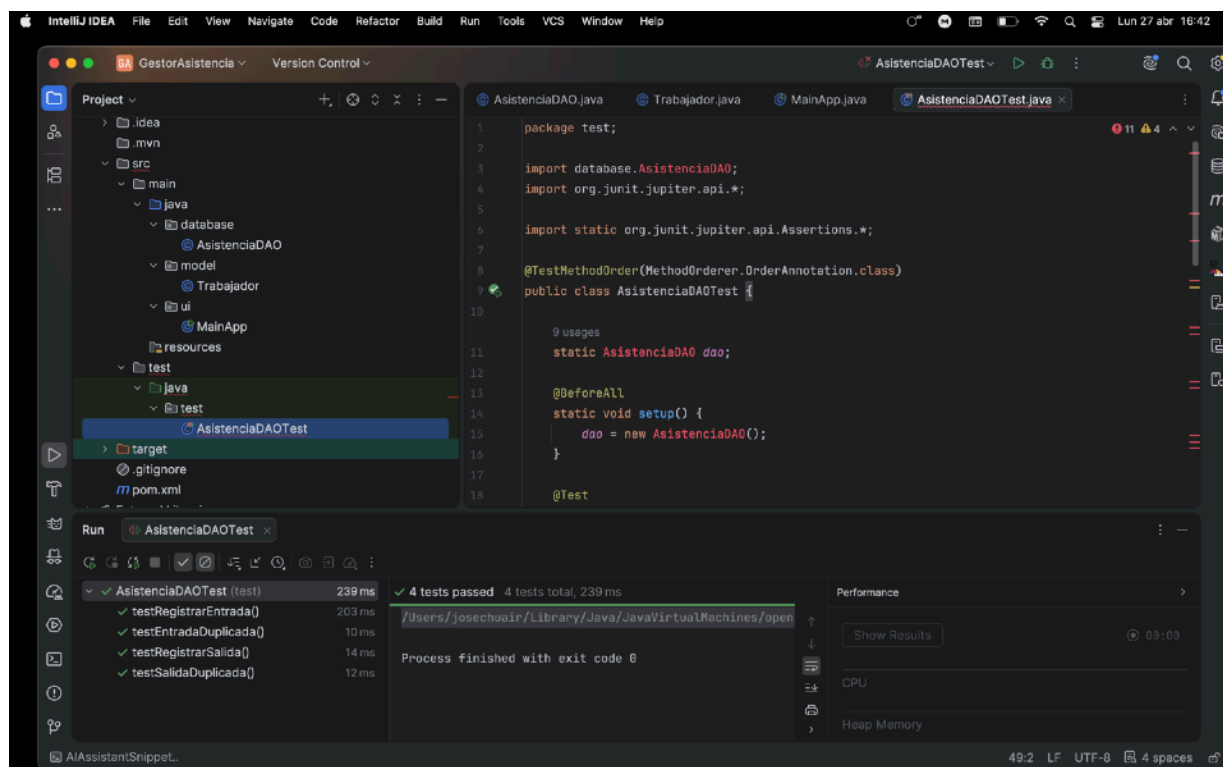
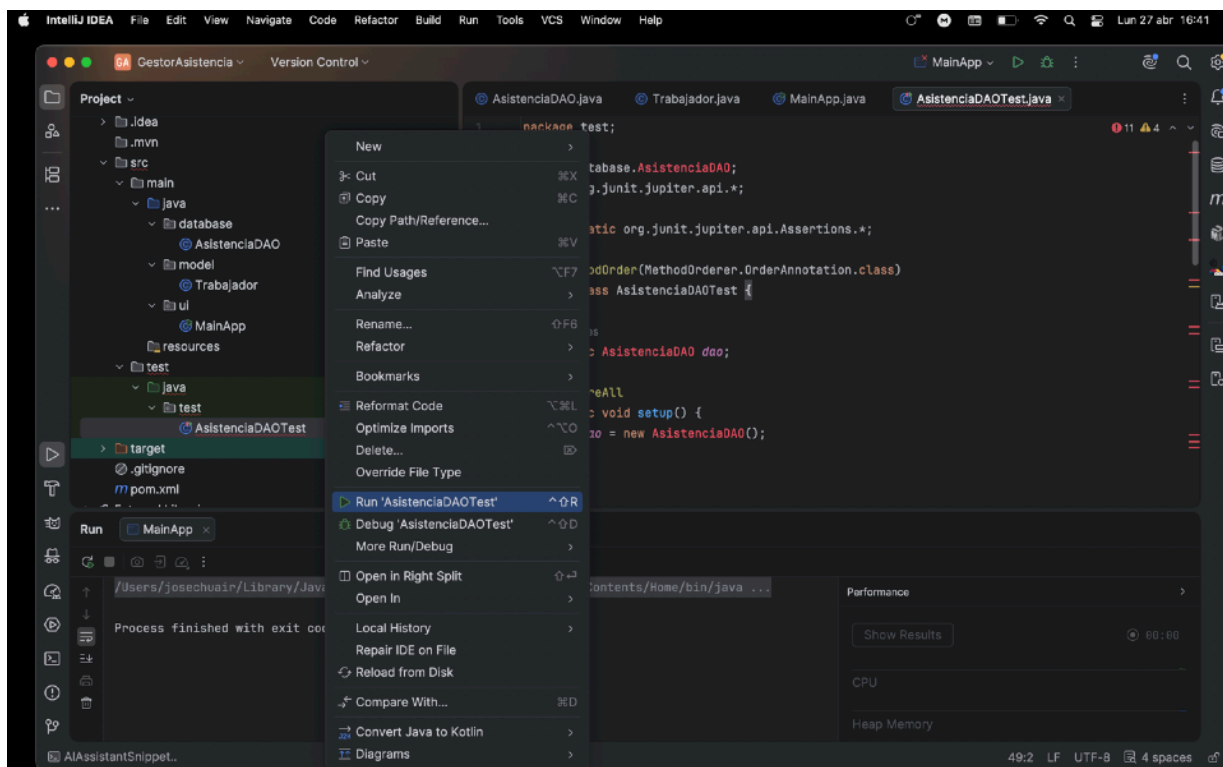
Verificar que un trabajador no puede registrar dos veces su salida.

Resultado esperado

El método debe devolver false .

Resultado obtenido

Correcto. La segunda salida fue rechazada.



6. Pruebas adicionales realizadas

Además de las pruebas obligatorias, se realizaron dos pruebas adicionales para mejorar la validación del sistema.

6.1 Prueba de seguridad (SQL Injection) Objetivo

Comprobar que la aplicación no es vulnerable a ataques de inyección SQL.

Procedimiento

Se introdujo en el campo nombre la siguiente cadena:

' OR '1'=1

Esta cadena es una técnica clásica utilizada para intentar alterar consultas SQL mal protegidas.

Resultado obtenido

La aplicación trató el valor como texto normal y registró correctamente la entrada sin alterar la consulta SQL ni comprometer la base de datos.

Conclusión

La aplicación está protegida frente a ataques básicos de inyección SQL gracias al uso de `PreparedStatement`, evitando la concatenación insegura de cadenas SQL.

6.2 Prueba de regresión Objetivo

Verificar que un cambio en la interfaz gráfica no afecta al funcionamiento interno del sistema.

Procedimiento

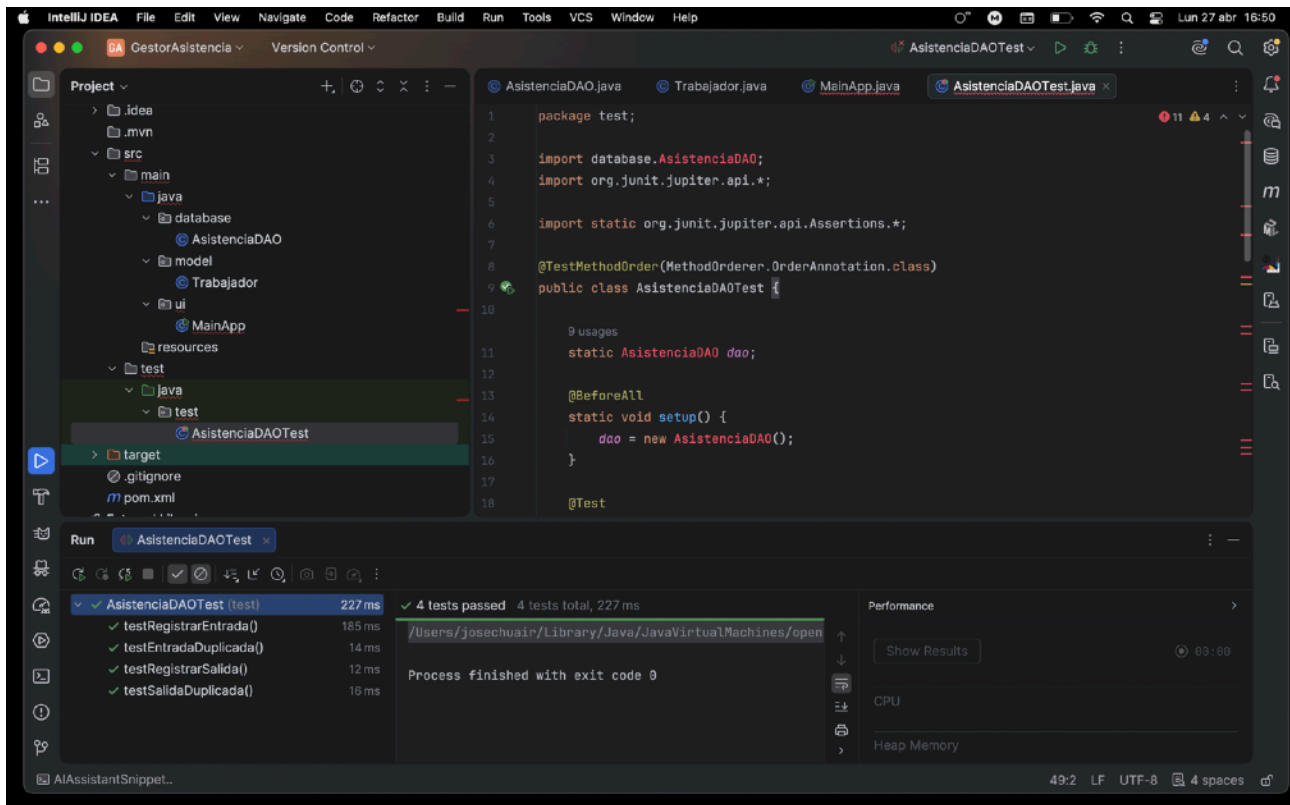
Se modificó el título principal de la interfaz: Antes:

Label titulo = `new` Label("Gestor de Asistencia"); Después:

Label titulo = `new` Label("Sistema de Control de Asistencia"); Posteriormente se ejecutaron nuevamente todas las pruebas unitarias con JUnit.

Resultado obtenido

Todos menos uno que salió en rojo salieron en verde, investigue y es por que tenia que borrar los datos de la tabla que estuve haciendo pruebas manuales con ella, los borro en con una sentencia SQL y los siguientes test ya siguieron funcionando correctamente y se mantuvieron en verde.



Conclusión

Los cambios visuales no afectan a la lógica del sistema ni a la conexión con la base de datos, validando correctamente la prueba de regresión.

7. Resultados obtenidos

Prueba

Inserción nueva entrada

Inserción nueva salida

Entrada duplicada

Salida duplicada

Seguridad SQL Injection

Prueba de regresión

Interfaz JavaFX

Conexión MySQL

JUnit

Resultado

Correcto

Correcto

Rechazada correctamente

Rechazada correctamente

Superada

Superada

Funcionando correctamente

Correcta

Tests superados

8. Problemas encontrados y soluciones aplicadas

Durante el desarrollo surgieron algunos problemas técnicos.

Error JavaFX runtime components are missing Problema

JavaFX no se ejecutaba directamente desde el método main.

Solución

Se configuró correctamente la VM de ejecución en IntelliJ añadiendo el module-path de JavaFX en Edit Configurations.

Error en pruebas JUnit por datos existentes Problema

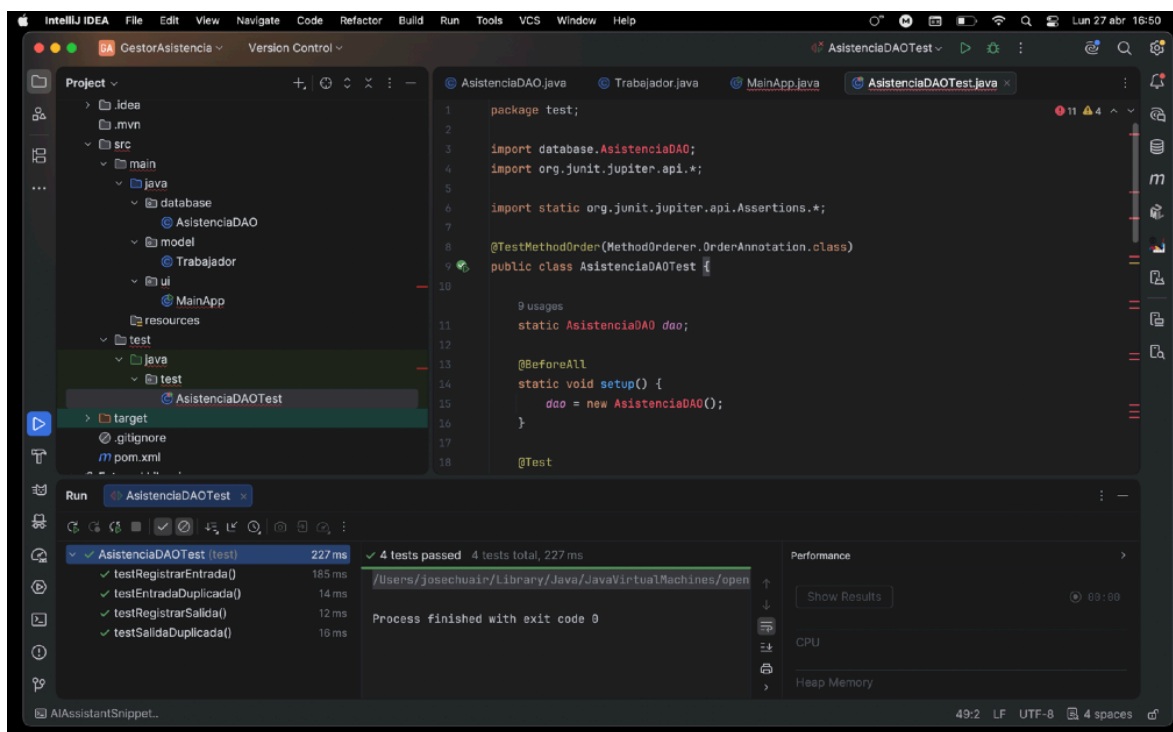
Algunas pruebas fallaban porque los datos de prueba ya existían en la base de datos.

Solución

Se vació la tabla asistencia mediante : `DELETE FROM asistencia;`



Después de limpiar la tabla, todas las pruebas se ejecutaron correctamente.



9. Conclusiones

El desarrollo de este caso práctico ha permitido implementar una aplicación real de control de asistencia con persistencia en base de datos y validación mediante pruebas automatizadas.

Se ha comprobado que:

- el sistema registra correctamente entradas y salidas • evita duplicidades
- mantiene la integridad de los datos
- presenta protección frente a inyección SQL
- soporta cambios en la interfaz sin afectar a la lógica
- las pruebas unitarias validan automáticamente el correcto funcionamiento

El uso de JavaFX mejora la experiencia de usuario al proporcionar una interfaz sencilla e intuitiva, mientras que MySQL permite almacenar la información de forma segura y persistente.

JUnit ha resultado fundamental para automatizar las pruebas y asegurar la estabilidad del sistema. Como mejora futura, podría añadirse:

- control de horarios
- gestión de múltiples empleados • panel de administración
- exportación de informes
- autenticación de usuarios

En conclusión, el proyecto cumple correctamente con los objetivos planteados y demuestra una aplicación práctica de desarrollo, pruebas y validación de software.

10. Bibliografía

Unidad 8 Interfaces

Apuntes Unidad 8

Documentación oficial de JavaFX

Documentación oficial de JUnit 5

Documentación MySQL Connector/J

Apache Maven Documentation